

State space reduced dynamic programming for the aircraft sequencing problem with constrained position shifting

F. Furini, M. P. Kidd, A. Persiani, P. Toth

Abstract

In this paper we present state space reduction techniques for a dynamic programming algorithm applied to the Aircraft Sequencing Problem (ASP) with Constrained Position Shifting (CPS). We consider the classical version of the ASP, which calls for determining the order in which a given set of aircraft should be assigned to a runway at an airport, subject to minimum separations in time between consecutive aircraft, in order to minimize the sum of the weighted deviations from the scheduled arrival/departure times of the aircraft. The focus of the paper is on a number of ways of improving the computation times of the dynamic programming algorithm proposed. This is achieved by using heuristic upper bounds and a completion lower bound in order to reduce the state space in the dynamic programming algorithm. We compare our algorithm to an approach based on mixed integer linear programming, which was adapted from the literature for the case of CPS. We show using real-world air traffic instances from the Milan Linate Airport that the dynamic programming algorithm significantly outperforms the MILP. Furthermore, we show that the proposed algorithm is capable of solving very large instances in short computation times, and that it is suitable for use in a real-time setting.

Keywords: Aircraft sequencing problem, dynamic programming, integer programming, completion bounds, constrained position shifting

1 Introduction

In this paper we consider the *aircraft sequencing problem* (ASP) [1, 2, 3, 4], which can be formally defined as follows. As input we are given the set A of aircraft that are scheduled to use the runway at an airport (either arriving or departing) during a certain time-window, and as output a time $x_i \in T$ should be assigned to each aircraft $i \in A$ for using the runway, as defined by a discrete set T of time instants. The assigned time x_i is restricted to fall within a time window $[e_i, \ell_i]$ for all $i \in A$, where $e_i \in T$ denotes the earliest time and $\ell_i \in T$ the latest time for i using the runway. Aircraft i is also associated with a scheduled (target) time $t_i \in T$ for using the runway, and the cost incurred by i for using the runway at some time $t \in T$ is given by

$$c(i, t) = g_i \max(0, t_i - t) + h_i \max(0, t - t_i),$$

where g_i and h_i are weights associated with i using the runway before and after its target time, respectively. If an aircraft $j \in A$ is scheduled to use the runway directly after an aircraft $i \in A$, it cannot use the runway earlier than $x_i + s_{ij}$, where s_{ij} denotes a minimum separation in time that has to be maintained between i and j for safety reasons.

We also consider so-called *Constrained Position Shifting* (CPS) [2, 5]. The motivation behind CPS is to ensure that a degree of fairness is maintained among the various airlines requesting usage of the runway, by restricting deviations from the first-come-first-serve (FCFS) sequence of the aircraft. Let f_i denote the position of aircraft $i \in A$ in the FCFS sequence of the aircraft (in which the elements of A are sorted according to increasing values of t_i for all $i \in A$). CPS restricts the position of aircraft $i \in A$ in the final sequence (in which the elements of A are sorted according to increasing values of x_i for all $i \in A$) to fall within the range $[f_i - p_i^-, f_i + p_i^+]$, where p_i^- denotes the *maximum position shift* of aircraft i to an earlier position in the sequence and p_i^+ to a later position in the sequence. CPS is often used in practice and the values of p_i^- and p_i^+ are usually equal to a small constant such as 1, 2 or 3 [3]. The objective of the ASP with CPS (ASP-CPS) is to determine a vector $\mathbf{x} = [x_i]$ of arrival/departure times for the aircraft so as to minimize the cost function

$$\sum_{i \in A} c(i, x_i),$$

subject to all the constraints mentioned above.

Preprocessing steps Some preprocessing steps may be performed in order to reduce the complexity of the problem without affecting the optimal solution. The first step is *time window tightening* [4]. Given an upper bound z_{UB} to the optimal solution value of the ASP, the size of the time window $[e_i, \ell_i]$ for aircraft $i \in A$ may be decreased by updating the values of e_i and ℓ_i to

$$e_i = \max[e_i, t_i - z_{UB}/g_i]$$

and

$$\ell_i = \min[\ell_i, t_i + z_{UB}/h_i].$$

The second step, called *position shift tightening*, is to determine a reduced set of positions to which an aircraft may be assigned in a feasible solution. Let P denote the set of ordered pairs of aircraft (i, j) where either $\ell_i < e_j + s_{ji}$ or $f_i + p_i^+ \leq f_j - p_j^-$. Hence P contains ordered pairs of aircraft (i, j) where i cannot use the runway after j without violating the maximum delay constraints, the maximum shift constraints or the minimum separation constraints. Furthermore, let $P_i^+ = |\{j : (i, j) \in P\}|$, *i.e.* the minimum number of aircraft using the runway after aircraft i in any feasible solution, and $P_i^- = |\{j : (j, i) \in P\}|$, the minimum number of aircraft using the runway before aircraft i in any feasible solution. A set of potential feasible positions for aircraft $i \in A$ may then be defined as

$$\mathcal{F}(i) = \{k : P_i^- < k < |A| - P_i^+ \text{ and } f_i - p_i^- \leq k \leq f_i + p_i^+\}.$$

In order to simplify notation in the discussions that follow, we further define

$$\mathcal{F}_{\min}(i) = \min[k : k \in \mathcal{F}(i)]$$

and

$$\mathcal{F}_{\max}(i) = \max[k : k \in \mathcal{F}(i)]$$

as the first and last possible positions, respectively, in which aircraft i may be scheduled in a feasible solution.

Literature review In this paper the focus is on exact approaches to the ASP. A survey of approaches and problems in the context of landing and take-off scheduling at airports was recently presented by Bennel *et al.* [6]. These problems have been subdivided into static and dynamic cases. In the static case it is assumed that complete information on the set of aircraft is known, while in the dynamic case the solution is revised each time new aircraft are introduced into the system. In this paper we consider the static case. Mixed integer linear programming (MILP) approaches are presented by Beasley *et al.* [4] and Abela *et al.* [7]. In [4] the authors consider the ASP for landings only and without considering CPS, while in [7] the authors proposed a MILP formulation for the dynamic ASP, and developed a branch-and-bound algorithm as a solution approach. An alternative formulation of the ASP was presented by Beasley and Pinol [8], where two objective functions were considered, one linear and one nonlinear. Dynamic programming approaches to the ASP-CPS were proposed by Psaraftis [9], Trivizas [10] and Balakrishnan and Chandran [3]. The algorithms presented in [9] and [10] considered the ASP-CPS without time windows, and in [9] it is assumed that all aircraft of the same type are identical. Balakrishnan and Chandran [3] present dynamic programming approaches for a number of different variants of the ASP-CPS, also providing complexity analyses of their algorithms.

Paper contribution We focus in particular on developing a fast dynamic programming algorithm for solving the ASP-CPS with average maximum position shift values. We propose a dynamic programming approach with a recursion which extends the one proposed in [3] for discrete time models with aircraft dependent cost functions (*i.e.* such as minimizing the weighted delay of the aircraft). However, in [3] the dynamic programming algorithm is solved over a so-called *CPS network*, whereas our dynamic programming algorithm is applied directly to the ASP. Furthermore, in [3] results are obtained for a different objective function and also for cases of the ASP with *precedence constraints* (pairs of aircraft with a fixed relative order), a constraint which is not strictly necessary in the real world cases considered in this paper. Furthermore, we propose a number of new ways by which the computation time required by the dynamic programming algorithm may be improved, such as using heuristic upper bounds and completion lower bounds in order to reduce the state space. These techniques enables us to consider larger values of the maximum shift compared to the ones considered in [3]. Finally we compare the improved dynamic programming approach to the MILP proposed in [4] by using instances from that paper as well as real-world instances from the Milan Linate Airport (proposed in [1]).

2 Mixed-integer programming formulation

A MILP formulation for the ASP was presented by Beasley *et al.* [4]. This formulation is extended in Model (1)–(11) below for the ASP-CPS. Here δ_{ij} is a binary decision variable such that

$$\delta_{ij} = \begin{cases} 1 & \text{if aircraft } i \in A \text{ uses the runway before aircraft } j \in A, \\ 0 & \text{otherwise,} \end{cases}$$

and α_i and β_i are decisions variables such that $\alpha_i = \max(0, t_i - x_i)$ and $\beta_i = \max(0, x_i - t_i)$ for aircraft $i \in A$. As mentioned before x_i is a decision variable denoting the time assigned to aircraft i for using the runway.

$$\text{Min} \quad \sum_{i \in A} (g_i \alpha_i + h_i \beta_i) \quad (1)$$

$$\text{s. t.} \quad e_i \leq x_i \leq \ell_i \quad i \in A \quad (2)$$

$$\delta_{ij} + \delta_{ji} = 1 \quad i, j \in A, i \neq j \quad (3)$$

$$x_i + s_{ij} - M(1 - \delta_{ij}) \leq x_j \quad i, j \in A, i \neq j \quad (4)$$

$$\alpha_i \geq t_i - x_i \quad i \in A \quad (5)$$

$$0 \leq \alpha_i \leq t_i - e_i \quad i \in A \quad (6)$$

$$\beta_i \geq x_i - t_i \quad i \in A \quad (7)$$

$$0 \leq \beta_i \leq \ell_i - t_i \quad i \in A \quad (8)$$

$$t_i - \alpha_i + \beta_i = x_i \quad i \in A \quad (9)$$

$$\mathcal{F}_{\min}(i) \leq \sum_{j \in A \setminus \{i\}} \delta_{ji} + 1 \leq \mathcal{F}_{\max}(i) \quad i \in A \quad (10)$$

$$\delta_{ij} \in \{0, 1\} \quad i, j \in A, i \neq j \quad (11)$$

where M is a large positive constant.

The objective function (1) aims at minimizing the total cost due to the weighted delay of the aircraft in the sequence. Constraints (2) ensure that the delay of each aircraft i respects its lower and upper bounds, while Constraints (3) ensure that either aircraft i uses the runway before aircraft j , or vice versa, for any two aircraft i and j . Constraints (4) imposes the minimum separation between aircraft while Constraints (5)–(9) ensure that $\alpha_i = \max(0, t_i - x_i)$ and $\beta_i = \max(0, x_i - t_i)$ holds. Finally, Constraints (10) ensure that the maximum position shift constraint is satisfied for all aircraft.

The parameter M in Constraints (4) may be replaced by $\ell_i + s_{ij} - e_j$ (as proposed in [4]), simplifying these constraints to

$$x_i + s_{ij} \delta_{ij} - (\ell_i - e_j) \delta_{ji} \leq x_j, \quad i, j \in A, i \neq j. \quad (12)$$

Constraints (4) may be strengthened by defining the sets described below. These sets were proposed in [4] only taking into account the maximum delay constraints. Here, however, the sets also take into account the maximum shift constraints.

Let P' be a subset of the set P (introduced in the previous section) such that it contains the ordered pairs of aircraft (i, j) where j may be delayed by i due to the minimum separation constraints, in other words where $f_i + p_i^+ \geq f_j - p_j^- - 1$ and $\ell_i + s_{ij} > e_j$. Furthermore, let Q denote the set of all ordered pairs of aircraft (i, j) such that neither (i, j) nor (j, i) are in P .

Constraints (4) may then be replaced by Constraints (13)–(15) below.

$$\delta_{ij} = 1 \quad (i, j) \in P \quad (13)$$

$$x_i + s_{ij} \leq x_j \quad (i, j) \in P' \quad (14)$$

$$x_i + s_{ij}\delta_{ij} - (\ell_i - e_j)\delta_{ji} \leq x_j \quad (i, j) \in Q \quad (15)$$

A number of valid inequalities were also proposed in [4], given by Constraints (16)–(18) below. Here Q' is the subset of Q where $t_i \leq t_j$ for any ordered pair $(i, j) \in Q$, and Q'' is the subset of Q' where $t_i + s_{ij} > t_j$ for any ordered pair $(i, j) \in Q'$.

$$\sum_{i \in A} \sum_{j \in A \setminus \{i\}} \delta_{ij} = |A|(|A| - 1)/2 \quad (16)$$

$$\delta_{ij} \geq 1 - \frac{\beta_i + \alpha_j}{t_j - t_i} \quad (i, j) \in Q' \quad (17)$$

$$(t_i + s_{ij} - t_j)\delta_{ij} + (t_j + s_{ji} - t_i)\delta_{ji} \leq \alpha_i + \beta_i + \alpha_j + \beta_j \quad (i, j) \in Q'' \quad (18)$$

3 Dynamic programming approach

As mentioned before, the recursion of the dynamic programming algorithm presented below is similar to expression (2) by Balakrishnan and Chandran [3]. In the following sections the recursion, the pseudocode of the algorithm, and the computation of bounds used for state pruning are discussed in detail.

3.1 Recursion

A state in the dynamic programming algorithm is defined by three parameters, namely a subset $S \subset A$ of aircraft, an aircraft $i \in A \setminus S$ such that $|S| + 1 \in \mathcal{F}(i)$ and a time instant $t \in [e_i, \ell_i]$. The state (S, i, t) represents the partial solution to the ASP-CPS where aircraft i uses the runway at time t after all of the aircraft in S have used the runway in an optimal way. The value of the state (S, i, t) , denoted by $\mathcal{D}(S, i, t)$, is therefore defined as the optimal solution value, if one exists, of the ASP-CPS when considering the set of aircraft $S \cup \{i\}$, and imposing the additional constraints $x_i = t$ and $x_j \leq x_i$ for all $j \in S$. If no such solution exists then $\mathcal{D}(S, i, t) = \infty$.

A recursion is easily established between the values of the states, namely

$$\mathcal{D}(S, i, t) = \min_{(i', t') \in S \times T : t \geq t' + s_{i', i}} [\mathcal{D}(S \setminus \{i'\}, i', t') + c(i, t)] \quad (19)$$

The set of pairs (i', t') considered in the minimization is, however, quite large and its size can be reduced in the following way. First of all, it is easy to verify that if aircraft i directly precedes aircraft j in the optimal sequence, then $x_j = x_i + s_{i,j}$ if $x_j > t_j$ (where x_j is the optimal time assigned for j to use the runway). Hence it will not be fruitful to consider values of t' such that

$t > t' + s_{i',i}$. Noting further that $t' \in [e_{i'}, \ell_{i'}]$ and $|S| \in \mathcal{F}(i')$ should hold, the set $S \times T$ can be substituted in (19) by the set $\mathcal{T}(S, i, t) \subseteq S \times T$ which, for each $i' \in S$ such that $|S| \in \mathcal{F}(i')$ and each $t' \in [e_{i'}, \ell_{i'}]$, contains the pair (i', t') if and only if

$$\begin{aligned} t &= t' + s_{i',i} & \text{if } t > t_i \\ t &\geq t' + s_{i',i} & \text{otherwise.} \end{aligned}$$

An improved recursion is therefore

$$\mathcal{D}(S, i, t) = \min_{(i', t') \in \mathcal{T}(S, i, t)} [\mathcal{D}(S \setminus \{i'\}, i', t') + c(i, t)]. \quad (20)$$

As initial values to this relation we have $\mathcal{D}(\emptyset, i, t_i) = 0$ if $1 \in \mathcal{F}(i)$, and by introducing a dummy aircraft i^* where $t_{i^*} = \max_{i \in I} \ell_i$ and $s_{ii^*} = 0$ for all $i \in A$, the optimal solution to the ASP-CPS for the input set A of aircraft is given by $\mathcal{D}(A, i^*, t_{i^*})$.

3.2 State space reduction techniques

By applying the following state space reduction techniques, it may be shown that some states in the set $\mathcal{T}(S, i, t)$ do not have to be considered in the recursion. This may be achieved by means of an upper bound z_{UB} to the optimal solution (as found, for instance, by a heuristic) and a *completion bound* $\mathcal{C}(S, i, t)$ for a state (S, i, t) . The completion bound is defined such that $\mathcal{D}(S, i, t) + \mathcal{C}(S, i, t)$ is a lower bound to the optimal solution value, if one exists, of the ASP-CPS when considering the set of aircraft I , and imposing the additional constraints $x_i = t$ and $x_j \leq x'_j$ for any $j \in S$ and $j' \in I \setminus S$. Hence a completion bound is a lower bound on the additional weighted delay which will be added to $\mathcal{D}(S, i, t)$ for any complete solution (S, i, t) may eventually lead to. Therefore, if $\mathcal{D}(S, i, t) + \mathcal{C}(S, i, t) > z_{UB}$, there is no need to consider the state (S, i, t) in the recursion.

Completion bound Given that some set S of aircraft uses the runway before aircraft $i \in A \setminus S$ does, let $\sigma(i, S) = \max(f_i - p_i^- - |S|, 1)$, i.e. $\sigma(i, S)$ represents the earliest possible position in which aircraft i can use the runway when taking the maximum shift constraints into consideration. The completion bound is then defined, for any state (S, i, t) , by

$$\mathcal{C}(S, i, t) = \sum_{j \in A \setminus (S \cup \{i\})} c(j, \max[t_j, t + s_{\min} \sigma(j, S \cup \{i\})]),$$

which is the sum of the weighted delays of the aircraft in $A \setminus S$ assuming that each one is assigned to its earliest possible position, where $s_{\min} = \min_{i,j \in A} s_{ij}$.

Heuristic upper bound In order to obtain an upper bound z_{UB} as discussed above we propose a tabu search heuristic. A solution in the tabu search heuristic is defined by a permutation of the given set of aircraft, and the runway times are calculated by assigning the aircraft to runway time slots in this order while taking the minimum separation constraints into consideration. A neighbour solution of a given feasible solution is defined as the solution obtained by performing a *swap*, i.e. exchanging the positions of two aircraft in the given solution. A swap may only be

performed if both aircraft still are in feasible positions in the neighbour solution (with respect to the maximum position shift p). During each iteration of the tabu search heuristic the tabu-list (updated in a first-in-first-out fashion) contains pairs of aircraft representing the swaps performed within a predefined number of previous iterations (the tabu *tenure*). The neighbour solutions of the current solution (initially the FCFS order) obtained by performing swaps not forbidden by the tabu list are evaluated in the order in which the aircraft appear in the current sequence. Then either the first neighbour that improves the current solution, or otherwise the overall best non-improving neighbour, replaces the current solution in the next iteration. The search continues until a predefined number of iterations is reached and the best overall solution found is given as output.

3.3 Dynamic programming algorithm

The pseudocode of the dynamic programming algorithm is given in Algorithms 1 (for initialization) and 2 (for the recursion). In Algorithm 1 preprocessing calculations are performed. This includes calculating the sets $\mathcal{F}(i)$ for each $i \in A$ in Step 5 and calculation of sets U_k and V_k for $2 \leq k \leq |A|$ in Steps 7–9. Here U_k is the set of all aircraft that can be assigned to position k and V_k is the set of aircraft that cannot be assigned to a position larger than k . These sets are later used in Steps 3–7 of Algorithm 2 in order to avoid states that cannot lead to any feasible solution. This is achieved by determining the set of all remaining aircraft which cannot be assigned to any position after the current one. If the cardinality of this set is larger than 1, then the current state cannot lead to a feasible solution and the state is pruned. If it is exactly equal to 1, then this aircraft has to be assigned the current position, and if the set is empty then it poses no additional restrictions. Finally in Steps 10–16 of Algorithm 1 the values of the initial states are set.

In Algorithm 2, a pre-existing state (S', i', t') is identified in Steps 1–2 and in Steps 3–11 all possible pairs (i, t) such that $(i', t') \in \mathcal{T}(S, i, t)$ where $S = S' \cup \{i'\}$ are determined. The algorithm then considers $\mathcal{D}(S', i', t') + c(i, t)$ as a new possible value for $\mathcal{D}(S, i, t)$ according to (20).

The completion bound is calculated in Step 15, and it is only necessary to do this once for each state independent of the value of the state. In order to achieve this the algorithm remembers whether a state has been considered or not. If $\mathcal{D}(S, i, t) + \mathcal{C}(S, i, t) \geq z_{UB}$ then this value of the state does not have to be considered in the algorithm, and the current value of the state is set to infinity instead in Step 19 of Algorithm 2 (yet allowing the state to attain a finite value again in future steps of the algorithm).

4 Computational experiments

For the purpose of performing computational experiments we consider three different sets of instances. The first set (presented in [1]) consists of 12 real air traffic instances of 60 aircraft each from Milan Linate Airport, the largest single runway airport in Italy. The second set consists of 7 instances of 10–44 aircraft presented in [4]. These two sets are used to compare

Algorithm 1: Initialization

Input : An input set A of aircraft and an upper bound z_{UB} .**Output** : The sets U_k , V_k and $\mathcal{F}(i)$, the values $\mathcal{D}(\emptyset, i, t_i)$ and updated values of e_i and ℓ_i .

```
1  $U_k \leftarrow \emptyset, V_k \leftarrow \emptyset$  for  $k = 2 \dots |A|$ ;  
2 for all  $i \in A$  do  
3    $e_i \leftarrow \max[e_i, t_i - z_{UB}/g_i]$ ;  
4    $\ell_i \leftarrow \min[\ell_i, t_i + z_{UB}/h_i]$ ;  
5   Determine  $\mathcal{F}(i)$ ;  
6   for all  $k \in \mathcal{F}(i)$  do  $U_k \leftarrow U_k \cup \{i\}$ ;  
7   ;  
8    $k \leftarrow$  the largest element of  $\mathcal{F}(i)$ ;  
9    $V_k \leftarrow V_k \cup \{i\}$ ;  
10  if  $1 \in \mathcal{F}(i)$  then  
11    for all  $t \in [e_i, t_i]$  do  
12       $\mathcal{D}(\emptyset, i, t) = c(i, t)$ ;  
13      Calculate completion bound  $\mathcal{C}(\emptyset, i, t)$ ;  
14      if  $\mathcal{D}(\emptyset, i, t) + \mathcal{C}(\emptyset, i, t) \geq z_{UB}$  then  $\mathcal{D}(\emptyset, i, t) = \infty$ ;  
15    end  
16  end  
17 end
```

Algorithm 2: Dynamic programming algorithm

Input : An input set A of aircraft with updated time windows, an upper bound z_{UB} , the sets U_k , V_k and $\mathcal{F}(i)$, and the values $\mathcal{D}(\emptyset, i, t_i)$.**Output** : The optimal arrival/departure time x_i of each aircraft $i \in A$.

```
1 for  $k = 2 \dots |A|$  do  
2   for all states  $(S', i', t')$  where  $|S'| = k - 2$  and  $\mathcal{D}(S', i', t') \neq \infty$  do  
3      $W \leftarrow A \setminus S' \cap U_k$ ;  
4     if  $|W \cap V_k| > 1$  then  $W \leftarrow \emptyset$ ;  
5     ;  
6     else if  $|W \cap V_k| = 1$  then  $W \leftarrow W \cap V_k$ ;  
7     ;  
8     for all  $i \in W$  do  
9        $t_{\min} = \max(t' + s_{i', i}, e_i)$ ;  
10       $t_{\max} = \min(t' + s_{i', i}, t_i)$ ;  
11      for  $t_{\min} \leq t \leq t_{\max}$  do  
12         $d = \mathcal{D}(S', i', t') + c(i, t)$ ;  
13         $S = S' \cup \{i'\}$ ;  
14        if  $(S, i, t)$  is considered for the first time then  
15          Calculate completion bound  $\mathcal{C}(S, i, t)$ ;  
16        end  
17        if  $d < \mathcal{D}(S, i, t)$  then  
18           $\mathcal{D}(S, i, t) = d$ ;  
19          if  $\mathcal{D}(S, i, t) + \mathcal{C}(S, i, t) \geq z_{UB}$  then  $\mathcal{D}(S, i, t) = \infty$ ;  
20        end  
21      end  
22    end  
23  end  
24 end
```

the performances of the dynamic programming algorithm and the MILP formulation. A direct comparison with the performance of the dynamic programming algorithm proposed in [3] for discrete time models is not possible, since in the computational section of that paper only results are presented on the version of the ASP which minimizes makespan. The third set consists of merging the 12 instances from Milan Linate Airport into a large sequence of 173 aircraft, and sampling from this sequence instances consisting of 100–160 aircraft. The third set is used to

show the effects of the heuristic upper bounds and the completion bound on the performance of the dynamic programming algorithm, and also to show the capability of this algorithm to solve large instances in very short computation times. These instances are larger than the realistic cases, and we also use a larger maximum shift value than that used in practice in order to enable the effects of using the various bounds to be more clear (since the computation times are small for realistic instances). In what follows $p_i^- = p_i^+ = p$ for all $i \in A$, as is customary in practice. All code was written in C in Linux Ubuntu on executed on an Intel(R) Core(TM)2 Duo CPU clocked at 2.33GHz with 2GB of RAM, and version 12.5 of CPLEX was used to solve the MILP. All computation times are presented in seconds.

In Tables 1 and 2 a comparison is made of the computation times and the number of states required by the dynamic programming algorithm when using various bounding procedures compared to the basic configuration where no pruning of states is done. For a heuristic upper bound we used either the FCFS value, or the value of the solution found by the tabu search with a tabu tenure of 50 and 1000 iterations. Results are also given with and without use of the completion bound. Furthermore, a memory limit of 1GB was imposed, and where this limit was exceeded no result is given. For $p = 6$ and $n \geq 120$ all the configurations of the dynamic programming algorithm exceeded the memory limit.

It can be seen from these results that in all cases the use of the completion bound improves the performance of the dynamic programming algorithm. Moreover, if the completion bound is not used, using the tabu search value as an upper bound slightly reduces the number of states, but does not reduce, on average, the computation time. However, using the tabu search and the completion bound together significantly outperforms the other configurations, especially for the largest instances. In fact, this configuration avoids exceeding the memory limit in more cases than the other configurations.

p	$ A $	No completion bound			With completion bound	
		No pruning	FCFS	Tabu search	FCFS	Tabu search
4	100	1.73	96%	116%	86%	76%
	110	2.06	96%	116%	89%	76%
	120	2.62	96%	114%	85%	72%
	130	3.00	95%	115%	87%	74%
	140	3.79	90%	108%	82%	68%
	150	4.36	90%	109%	82%	69%
	160	4.68	95%	114%	86%	74%
5	100	9.65	94%	90%	82%	47%
	110	11.49	94%	91%	85%	48%
	120	14.45	95%	91%	83%	46%
	130	16.60	95%	91%	84%	48%
	140	19.81	94%	90%	83%	47%
	150		21.41	96%	88%	50%
	160			23.01	93%	53%
6	100				31.39	65%
	110					25.96
	120					29.75

Table 1: Results on the average time required by the exact dynamic programming algorithm using various bounding procedures. In each row the left-most value is given in seconds, while the preceding values in that row is given as a percentage of the first value. For each value of p and $|A|$, five instances were considered, and in cases where the algorithm exceeded the memory limit no result is given.

p	$ A $	No completion bound			With completion bound	
		No pruning	FCFS	Tabu search	FCFS	Tabu search
4	100	182795.8	96%	89%	75%	41%
	110	201753.2	97%	90%	78%	43%
	120	235135.4	96%	90%	75%	40%
	130	252977.6	96%	90%	78%	42%
	140	285062.6	96%	89%	77%	41%
	150	308930.4	96%	90%	77%	41%
	160	328962.8	96%	89%	77%	42%
5	100	867619.6	96%	87%	73%	36%
	110	948986.8	97%	89%	77%	39%
	120	1116559.8	96%	88%	74%	36%
	130	1200858.2	96%	88%	76%	39%
	140	1349463.6	96%	88%	75%	37%
	150		1399297.4	92%	79%	40%
	160			1358824.0	87%	51%
6	100				2098792.0	66%
	110					1656792.8
	120					1725284.0

Table 2: Results on the average number of states required by the exact dynamic programming algorithm using various bounding procedures. In each row the left-most value is given as a number of states, while the preceding values in that row is given as a percentage of the first value. For each value of p and $|A|$, five instances were considered, and in cases where the algorithm exceeded the memory limit no result is given.

As mentioned before, realistic values of p are in the range from 1 to 3, and so for the comparison of the two exact approaches these values are used. In Table 3 the required computation times for the dynamic programming algorithm and the MILP formulation are given for each of the real-world instances we consider.

For the instances from [4] (BKSA) both algorithms require low computation times. This behaviour may be attributed to the fact that these instances consist of relatively small numbers of aircraft. However, in the majority of these cases the dynamic programming algorithm is able to outperform the MILP formulation. For the real-world Milan Linate Airport instances from [1] (FPT), the dynamic programming algorithm outperforms the MILP formulation by a significant margin. The computation times vary much for the MILP formulation, however for the dynamic programming algorithm the times are consistent, and increases as the maximum shift value increases. This is to be expected as the complexity of the dynamic programming approach increases as the maximum shift value increases.

5 Conclusion

In this paper we proposed a dynamic programming algorithm which is capable of solving real-world air traffic instances in very short computation times, making it suitable for use in a real-time setting. We were able to utilize heuristic upper bounds and a completion lower bound in order to speed up the performance of the dynamic programming algorithm, and showed that this enabled us to solve very large instances in short computation times. Moreover, we compared the performance of the algorithm to that of a MILP formulation and concluded that it is a more effective approach to use for the real-world instances we considered.

Instance	A	$p = 1$		$p = 2$		$p = 3$	
		DP	MILP	DP	MILP	DP	MILP
BKSA1	10	0.00	0.01	0.01	0.02	0.03	0.02
BKSA2	15	0.02	0.02	0.03	0.13	0.13	0.16
BKSA3	20	0.04	0.02	0.17	0.04	0.52	0.03
BKSA4	20	0.03	0.11	0.12	0.91	0.39	3.73
BKSA5	20	0.03	0.13	0.11	4.92	0.31	23.93
BKSA6	30	0.00	0.00	0.01	0.01	0.03	0.01
BKSA7	44	0.03	0.20	0.08	0.27	0.08	0.27
FPT1	60	0.00	0.85	0.08	257.44	0.17	27.59
FPT2	60	0.00	1.12	0.07	TL	0.16	TL
FPT3	60	0.00	0.74	0.07	866.89	0.17	1153.95
FPT4	60	0.00	5.57	0.08	2396.44	0.17	324.79
FPT5	60	0.00	1.90	0.08	175.61	0.18	496.18
FPT6	60	0.00	0.48	0.08	9.73	0.17	12.58
FPT7	60	0.00	0.55	0.08	2.33	0.17	12.80
FPT8	60	0.00	0.37	0.08	2.28	0.17	2.51
FPT9	60	0.00	0.26	0.08	1.53	0.18	3.25
FPT10	60	0.00	0.53	0.08	1.33	0.18	3.90
FPT11	60	0.00	0.64	0.08	11.60	0.18	107.23
FPT12	60	0.00	0.14	0.08	0.91	0.18	1.88

Table 3: The required computation times for the dynamic programming (DP) algorithm and the MILP formulation where $p = 1, 2, 3$, for the 7 instances from [4] (BKSA) and the 12 instances from [1] (FPT). For the MILP formulation a time limit of one hour was imposed, and the time limit was reached in these two cases indicated by ‘TL.’

For future work it could be fruitful to investigate, both from a theoretical and computational point of view, the effects on the dynamic programming algorithm of larger time windows, larger maximum shift values, and also maximum shift values that vary among the aircraft.

References

- [1] F Furini, C A Persiani, and P Toth. Aircraft sequencing problems via a rolling horizon algorithm. *Lecture Notes in Computer Science*, 7422:273–284, 2012.
- [2] R G Dear and Y S Sherif. The dynamic scheduling of aircraft in high density terminal areas. *Microelectronics Reliability*, 29(5):743–749, 1989.
- [3] H Balakrishnan and B G Chandran. Algorithms for scheduling runway operations under constrained position shifting. *Operations Research*, 58(6):1650–1665, 2010.
- [4] J.E. Beasley, M. Krishnamoorthy, Y.M. Sharaiha, and D. Abramson. Scheduling aircraft landing — the static case. *Transportation Science*, 34:180–197, 2000.
- [5] R. de Neufville and A. R. Odoni. *Airport Systems: Planning, Design and Management*. McGraw-Hill, New York, 2003.
- [6] J.A. Bennel, M. Mesgarpour, and C.N. Potts. Airport runway scheduling. *4OR*, 9:115–138, 2011.
- [7] J. Abela, D. Abramson, M. Krishnamoorthy, A. De Silva, and G. Mills. Computing optimal schedules for landing aircraft. *Proceeding 12th National ASOR Conference, Adelaide, Australia*, pages 71–90, 1993.

- [8] J.E. Beasley and H. Pinol. Scatter search and bionomic algorithms for the aircraft landing problem. *European Journal of Operational Research*, 127(2):439–462, 2006.
- [9] H. N. Psaraftis. A dynamic programming approach for sequencing groups of identical jobs. *Operations Research*, 28(6):1347–1359, 1980.
- [10] D. A. Trivizas. Optimal scheduling with maximum position shift (mps) constraints: A runway scheduling application. *Journal of Navigation*, 51(2):250–266, 1998.